



Leadwise

AI CRM Dashboard

MERN stack project: setup and explanation

Rohit Sharma, Sharma Textiles



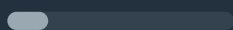
87

Neha Joshi, Pune EdTech



54

Manish Tiwari, Tiwari Caterers



18

What is inside

1. Project overview and features
2. Technology stack and architecture
3. Step-by-step setup and run guide
4. How to use the application
5. Database design (MongoDB)
6. How the AI works
7. API reference
8. Troubleshooting
9. Viva questions and answers

Stack: MongoDB · Express.js · React · Node.js · Google Gemini / OpenAI (optional)

1. Project overview

Leadwise is a web-based Customer Relationship Management (CRM) system for small and medium businesses. A CRM stores information about potential customers (called **leads**), tracks where each one is in the sales process, and reminds the sales team what to do next.

Most small businesses track leads in notebooks, WhatsApp chats or Excel sheets. Good leads get forgotten, follow-ups are missed, and nobody knows which customer to call first. This project solves that problem and adds **Artificial Intelligence** to rank leads, read the mood of call notes, write emails and answer questions about sales data.

Problem statement

To design and develop a full-stack web application using the MERN stack that manages leads, sales pipeline and tasks, and uses AI to score leads by their chance of converting, analyse note sentiment, generate sales emails, forecast revenue and provide a conversational assistant.

Objectives

- Store and manage leads, notes and tasks securely for each user.
- Show the sales pipeline visually as a dashboard and a drag-and-drop board.
- Score every lead from 0 to 100 using AI and explain the reasons.
- Detect positive, neutral or negative sentiment in call notes.
- Generate professional emails automatically.
- Forecast expected revenue from the open pipeline.
- Work even without internet or an API key, using a built-in offline AI engine.

Main features

Module	What it does
Login and sign-up	Secure accounts with JWT tokens and encrypted (hashed) passwords.
Dashboard	AI revenue forecast, open leads, won value, win rate, 6-month trend chart, pipeline by stage, AI insights, top leads, upcoming tasks and activity feed.
Leads	Add, edit, delete, search and filter leads. Sort by AI score or value. Export to CSV (Excel).
Lead detail	Stage bar, AI score with reasons and next step, notes with mood tags, AI email writer, tasks and history.
Pipeline	Kanban board. Drag a lead card to move it to another stage.
Tasks	Calls, emails and meetings linked to leads, grouped into overdue, today and later.
AI assistant	Chat that answers questions such as "Who should I call today?" using live CRM data.

2. Technology stack and architecture

MERN stands for the four main technologies. All of them use JavaScript, so one language runs the whole project.

Layer	Technology	Role in this project
Database	MongoDB + Mongoose	Stores users, leads, tasks and activity as JSON-like documents.
Backend	Express.js on Node.js	REST API: login, lead and task operations, dashboard numbers, AI features.
Frontend	React 18 + Vite	The user interface that runs in the browser.
Charts	Recharts	Trend chart and pipeline bar chart on the dashboard.
Security	JWT + bcryptjs	Login tokens and password hashing.
AI	Gemini / OpenAI (optional) + offline engine	Lead scoring, sentiment, emails, chat assistant.

How the parts connect

```
Browser (React app, port 5173)
  | HTTP requests with JWT token (Axios)
  v
Express API (Node.js, port 5000)
  |-- auth middleware -> checks the token
  |-- routes          -> leads, tasks, dashboard, ai
  |-- aiService       -> Gemini/OpenAI or offline engine
  |   | Mongoose
  |   v
MongoDB database (local or Atlas cloud)
```

Example flow: when the user adds a note, React sends it to **POST /api/leads/:id/notes**. The backend checks the login token, asks the AI service for the note's sentiment, saves the note in MongoDB, re-calculates the lead's AI score, logs an activity entry and sends the updated lead back. React then refreshes the screen.

Folder structure

```
ai-crm-mern/
|-- backend/
|   |-- config/db.js           MongoDB connection
|   |-- middleware/auth.js     JWT login check
|   |-- models/               User, Lead, Task, Activity schemas
|   |-- routes/               auth, leads, tasks, dashboard, ai
|   |-- services/aiService.js  All AI logic
|   |-- services/stats.js      Dashboard numbers and forecast
|   |-- seed.js               Loads demo data
|   |-- server.js              Starts the API
|   |-- .env.example           Settings template
|-- frontend/
|   |-- src/components/        Layout, ScoreBar, Modal, LeadForm ...
|   |-- src/pages/             Dashboard, Leads, LeadDetail, Pipeline,
|   |                           Tasks, Assistant, AuthPage
|   |-- src/context/           Login state (AuthContext)
|   |-- src/api.js             Axios setup and helpers
|   |-- src/index.css          All styling
|-- README.md
```

3. How to run the project (step by step)

These steps are written for Windows. On macOS or Linux, the commands are the same except where noted. You need an internet connection only for installing packages the first time.

1 Install the required software

Software	Where to get it	Check it works
Node.js (version 18 or newer, LTS)	nodejs.org	node -v
MongoDB Community Server	mongodb.com/try/download/community (tick "Install MongoDB as a Service" and MongoDB Compass)	Open MongoDB Compass and click Connect
VS Code (recommended)	code.visualstudio.com	Open the project folder in it

No MongoDB on the laptop? Use the free cloud database instead: create an account at mongodb.com/atlas, create a free M0 cluster, add a database user, allow access from anywhere (0.0.0.0/0) under Network Access, and copy the connection string. You will paste it in Step 4.

2 Unzip the project

Extract **ai-crm-mern.zip** to a simple path such as **D:\projects\ai-crm-mern**. Open that folder in VS Code, then open a terminal with **Terminal → New Terminal**.

3 Install backend packages

```
cd backend
npm install
```

This downloads Express, Mongoose and the other libraries into a node_modules folder. It takes 1-2 minutes.

4 Create the settings file (.env)

```
copy .env.example .env      (Windows)
cp .env.example .env        (macOS / Linux)
```

Open **backend/.env** in VS Code and check these values:

Setting	What to put
MONGO_URI	Keep mongodb://127.0.0.1:27017/ai_crm for local MongoDB, or paste your Atlas connection string.
JWT_SECRET	Any long random text, e.g. leadwise_final_year_2026_secret_key
GEMINI_API_KEY	Optional. Leave empty to use the offline AI engine. For real AI, get a free key at aistudio.google.com/app/apikey and paste it here.

5 Load demo data

```
npm run seed
```

Expected output: **Seeded 24 leads and 14 tasks. Login: demo@crm.com / demo123.**
This creates a demo account filled with sample Indian business leads, so the dashboard has data to show.

6 Start the backend server

```
npm run dev
```

Expected output: **MongoDB connected** and **API running on http://localhost:5000.**
Keep this terminal open.

Test it: open <http://localhost:5000/api/health> in the browser. It should show `{"ok":true}`.

7 Install and start the frontend

Open a **second terminal** (the + button in the VS Code terminal panel) and run:

```
cd frontend
npm install
npm run dev
```

Expected output: **Local: http://localhost:5173/**

8 Open the app and sign in

Open **http://localhost:5173** in Chrome. The login form is already filled with the demo account:

Email: demo@crm.com **Password:** demo123

Click **Sign in**. The dashboard opens.

Every time you run it again later

You only need Steps 6 and 7 (without npm install): start the backend with **npm run dev** in the backend folder, and the frontend with **npm run dev** in the frontend folder, in two separate terminals.

Optional: single-server production build

```
cd frontend
npm run build
cd ../backend
npm start           (app and API both on http://localhost:5000)
```

4. How to use the application

A suggested order for a live demo or viva presentation:

- **Dashboard.** Show the AI forecast, charts, the "What the AI noticed" panel and the "Call these first" list.
- **Leads.** Search for a name, filter by Hot leads, then click **Add lead** and create a new one. It gets an AI score instantly.
- **Lead detail.** Add a note such as "Very interested, budget approved". It is tagged positive and the score goes up. Then add "Says the price is too expensive" and show the score going down.
- **AI email.** Choose Proposal and Formal, click **Write email**, then Copy or Open in mail app.
- **Pipeline.** Drag a card from Qualified to Proposal. The score updates.
- **Tasks.** Add a task with a due date and tick it as done.
- **AI assistant.** Click the ready-made questions, e.g. "Who should I call today?" and "What is my revenue forecast?"

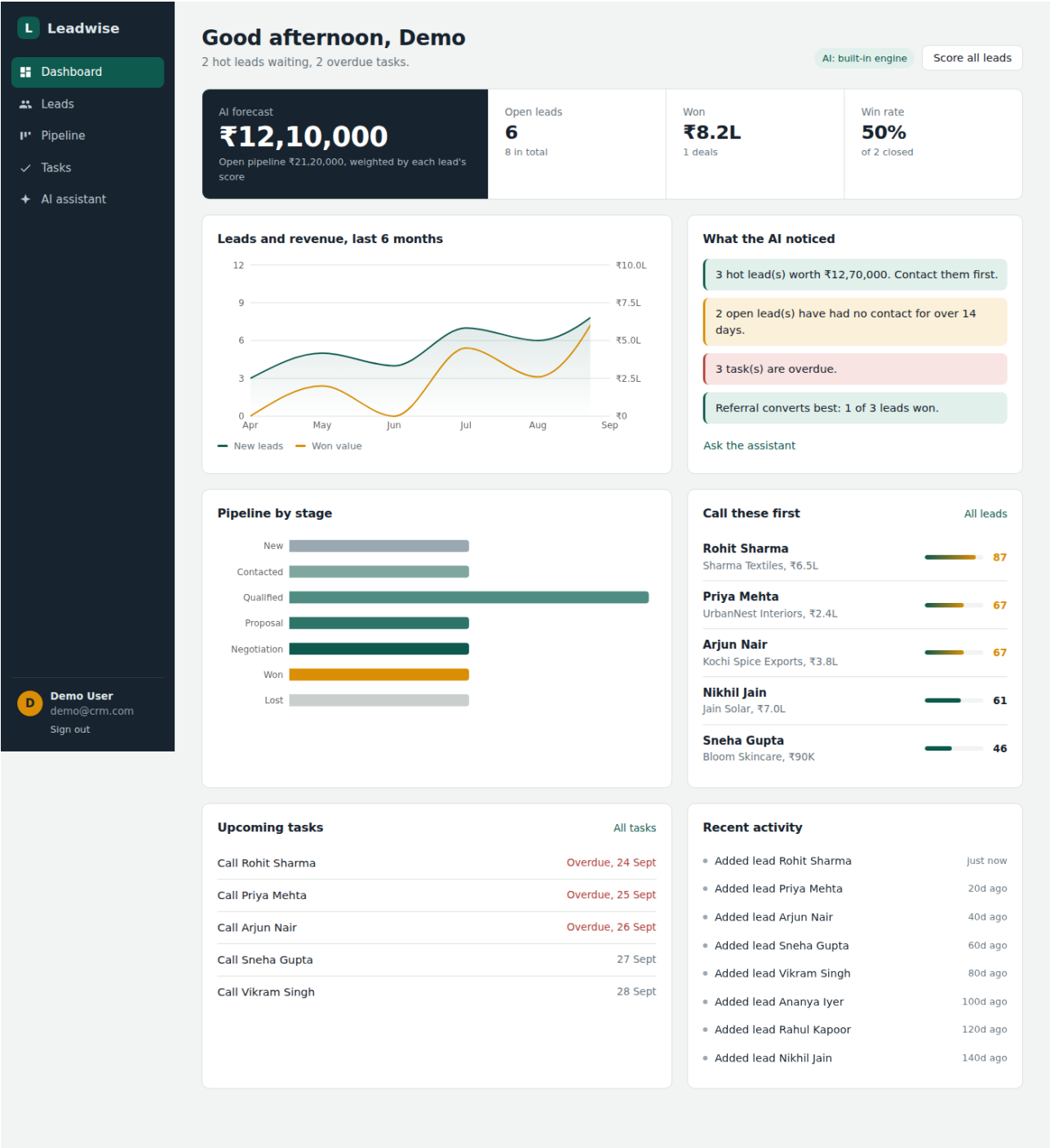


Figure 1: Dashboard with AI forecast, charts, insights and top leads

L

Leadwise

Dashboard

Leads

Pipeline

Tasks

AI assistant

D

Demo User

demo@crm.com

Sign out

Leads

Rohit Sharma

Owner at Sharma Textiles

EditDelete

NewContactedQualifiedProposalNegotiationWonLost

AI lead score

Score again

87

Hot

Next step: Share the contract and agree on a start date.

• Stage "Negotiation" (+33)

• High budget (+16)

• Source Referral (+12)

• Deal value above ₹5L (+10)

• Talking to a decision maker (+8)

• Full contact details (+4)

• 1 positive note(s)

Scored just now by the built-in engine

Notes

What happened on the call? The AI tags the mood and updates the score.

Add note

PositiveVery interested in the demo

Just now

Write an email with AI

Follow-up

Friendly

Write email

Details

Deal value₹6,50,000

BudgetHigh

SourceReferral

Emaila@b.in

Phone+91 9800000000

Last contact-

Added26 Sept

Tasks

Call about pricing

mm/dd/yyyy

Add

Call Rohit Sharma

24 Sept

Call Priya Mehta

25 Sept

History

Figure 2: Lead detail page with the AI score, reasons, notes and email writer

L Leadwise

Dashboard

Leads

Pipeline

Tasks

AI assistant

Demo User

demo@crm.com

Sign out

Leads

8 shown, sorted by AI score

Export CSV

Add lead

Search name, company, email or phone

All stages

All sources

Hot, warm and co

Highest AI score

Lead	Stage	Source	Value	AI score	Updated
Rohit Sharma Owner, Sharma Textiles	Negotiation	Referral	₹6,50,000	87	just now
Priya Mehta Founder, UrbanNest Interiors	Proposal	Google Ads	₹2,40,000	67	3d ago
Arjun Nair Director, Kochi Spice Exports	Qualified	LinkedIn	₹3,80,000	67	6d ago
Sneha Gupta Marketing Manager, Bloom Skincare	Contacted	Meta Ads	₹90,000	46	9d ago
Vikram Singh CEO, Singh Logistics	Won	Event	₹8,20,000	100	12d ago
Ananya Iyer Owner, Chennai Dental Care	New	Website	₹60,000	24	15d ago
Rahul Kapoor Partner, Kapoor Jewellers	Lost	Website	₹2,00,000	10	18d ago
Nikhil Jain CEO, Jain Solar	Qualified	Google Ads	₹7,00,000	61	21d ago

Figure 3: Leads list with filters and the AI score column

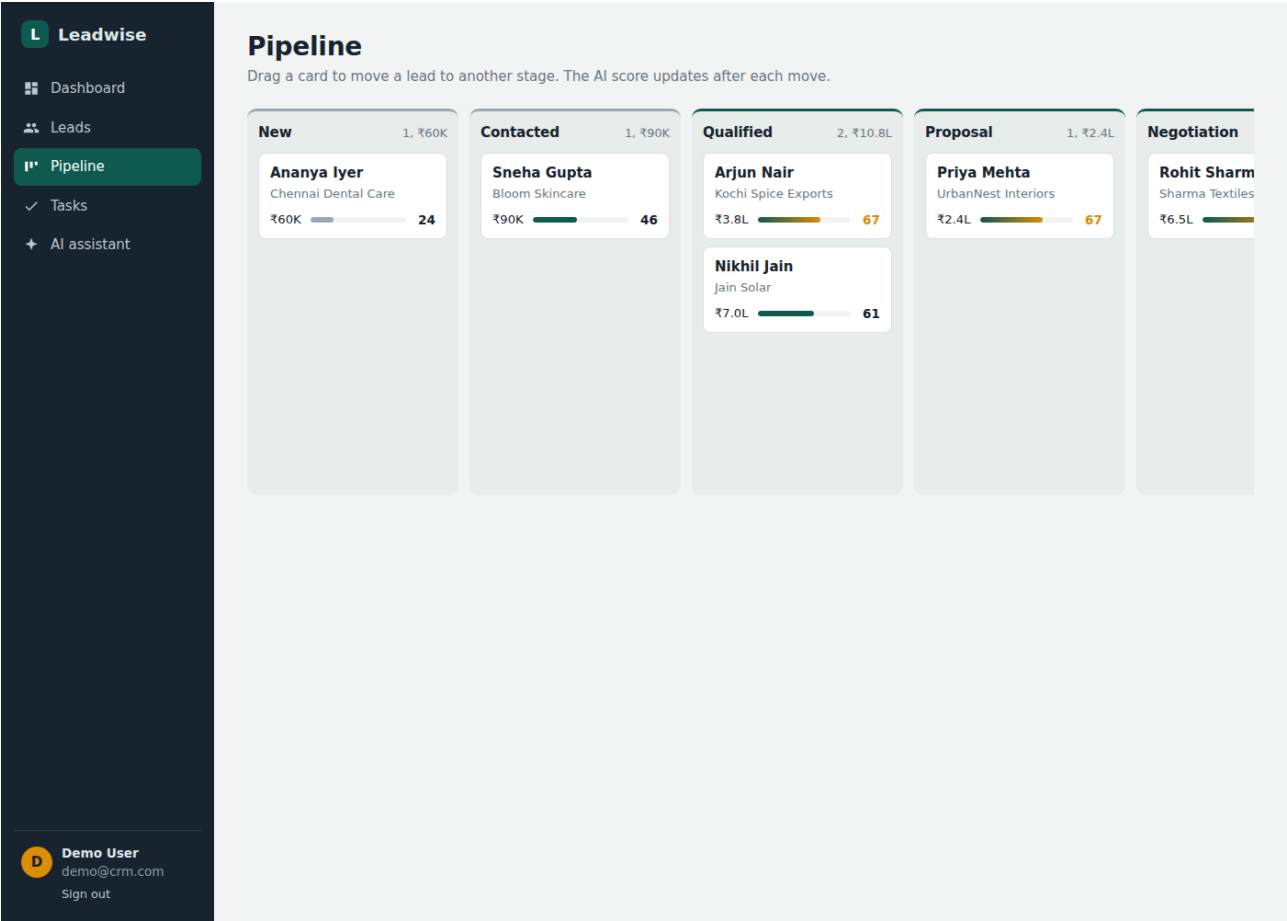


Figure 4: Pipeline board with drag and drop

L

Leadwise

Dashboard

Leads

Pipeline

Tasks

AI assistant

D

Demo User

demo@crm.com

Sign out

Tasks

Calls, emails and meetings to keep deals moving.

To do

Completed

All

What needs doing?

Call

No lead

Medium

mm/dd/yyyy

Add task

Overdue 2

☐

Call Rohit Sharma

Call for Rohit Sharma, high priority

24 Sept

X

☐

Call Priya Mehta

Call for Priya Mehta, medium priority

25 Sept

X

Today 1

☐

Call Arjun Nair

Call for Arjun Nair, low priority

26 Sept

X

Later 1

☐

Call Sneha Gupta

Call for Sneha Gupta, high priority

27 Sept

X

Figure 5: Tasks grouped by due date

Screenshots were taken with sample data. Numbers will differ on your machine.

5. Database design (MongoDB)

Yes, the project uses a database: **MongoDB**, a NoSQL database that stores data as documents (similar to JSON). Mongoose defines a schema for each collection so the data stays consistent. The database is named **ai_crm** and has four collections. You can see them in MongoDB Compass after running the seed script.

users

Field	Type	Notes
name, email	String	Email is unique
password	String	Hashed with bcrypt, never stored as plain text
role	String	admin or sales
company	String	
createdAt, updatedAt	Date	Added automatically

leads

Field	Type	Notes
owner	ObjectId → users	Each user only sees their own leads
name, email, phone, company, jobTitle	String	Contact details
source	String	Website, Referral, Google Ads, Meta Ads, LinkedIn, Cold Call, Event, Other
status	String	New → Contacted → Qualified → Proposal → Negotiation → Won / Lost
value, budget	Number, String	Deal value in ₹ and budget level
notes[]	Array of sub-documents	text, sentiment (positive / neutral / negative), date
ai	Object	score, label (Hot/Warm/Cold), reasons[], nextAction, provider, scoredAt
lastContactedAt	Date	Used to find leads going cold

tasks

Field	Type	Notes
owner	ObjectId → users	
lead	ObjectId → leads	Optional link to a lead
title, type	String	Call, Email, Meeting, Follow-up, Other
dueDate, priority, done	Date, String, Boolean	

activities

Field	Type	Notes
owner, lead	ObjectId	
kind	String	created, status, note, ai, task, email
message	String	e.g. "Rohit Sharma: Proposal → Negotiation"

Relationships

One user has many leads, tasks and activities. One lead has many notes (stored inside the lead), tasks and activities. The **owner** field on every record makes sure users can never read or change another user's data.

6. How the AI works

All AI logic is in **backend/services/aiService.js**. It runs in one of two modes:

Mode	When	How
LLM mode	GEMINI_API_KEY or OPENAI_API_KEY is set in .env	Sends a prompt with the lead or CRM data to Google Gemini or OpenAI and reads the answer.
Offline mode	No key, no internet, or the LLM call fails	A built-in rule-based engine. Always works, so the demo never breaks.

6.1 Lead scoring

The offline engine adds and subtracts points, then limits the result to 0-100:

Factor	Points
Pipeline stage	New 5, Contacted 12, Qualified 22, Proposal 28, Negotiation 33, Won 40
Budget	Low 4, Medium 10, High 16
Lead source	Referral 12, Event 8, LinkedIn / Google Ads 7, Website 6, Meta Ads 5, Cold Call 2
Deal value	Above ₹5 lakh +10, above ₹1 lakh +6, any value +3
Decision maker (CEO, Owner, Director, Founder, Manager ...)	+8
Both email and phone given	+4
Positive notes	+4 each (max +10)
Negative notes	-5 each (max -15)
No contact for more than 14 days	-10

Labels: 65 or more is **Hot**, 40 to 64 is **Warm**, below 40 is **Cold**. Each score is saved with its reasons, so the AI is explainable: the user can see exactly why a lead scored high or low. The engine also recommends a next step based on the stage.

In LLM mode, the lead details and last five notes are sent to the model together with the offline score as a reference. The model replies in JSON with a score, label, reasons and next action.

6.2 Sentiment analysis

Offline, each note is checked against lists of positive words (interested, approved, ready, demo ...) and negative words (expensive, no budget, delay, competitor ...). Phrases like "not interested" are handled correctly as negative. In LLM mode the model classifies the note directly.

6.3 Revenue forecast

The forecast treats each open lead's AI score as a probability of winning:

Forecast = Σ (deal value $\times$ AI score $\div$ 100) for all open leads.

Example: a ₹6,50,000 lead with score 87 adds ₹5,65,500. A ₹60,000 lead with score 24 adds ₹14,400.

6.4 Email writer and assistant

The email writer fills professional templates using the lead's name, company and value (offline), or asks the LLM to write a short personalised email. The assistant builds a snapshot of the user's leads, tasks and statistics. Offline, it recognises common questions (hot leads, forecast, pipeline, overdue tasks, cold leads, best source, win rate). In LLM mode it can answer any question about that data.

7. API reference

Base URL: <http://localhost:5000/api>. All routes except register and login need the header **Authorization: Bearer <token>**.

Method	Route	Purpose
POST	/auth/register	Create an account
POST	/auth/login	Sign in, returns a token
GET	/auth/me	Current user
GET	/leads	List leads (search, status, source, label, sort)
POST	/leads	Create a lead (auto-scored)
GET	/leads/:id	One lead with tasks and history
PUT	/leads/:id	Update a lead
PATCH	/leads/:id/status	Change stage (pipeline)
DELETE	/leads/:id	Delete a lead and its tasks
POST	/leads/:id/notes	Add a note (auto sentiment)
GET / POST	/tasks	List or create tasks
PUT / DELETE	/tasks/:id	Update or delete a task
GET	/dashboard	Stats, insights, top leads, activity
GET	/ai/status	Which AI mode is active
POST	/ai/score/:id	Re-score one lead
POST	/ai/score-all	Re-score all leads
POST	/ai/email	Write an email
POST	/ai/chat	Ask the assistant
POST	/ai/sentiment	Check sentiment of any text

8. Troubleshooting

Problem	Fix
"Could not connect to MongoDB"	MongoDB is not running. Open Services (Windows key, type services), find MongoDB Server and click Start. For Atlas, check the connection string password and that Network Access allows your IP.
"npm is not recognized"	Node.js is not installed or VS Code was open during install. Install Node.js LTS and restart VS Code.
Demo login says "Email or password is incorrect"	Run npm run seed in the backend folder, then try again.
App shows "Could not reach the server"	The backend is not running. Start it with npm run dev in the backend folder.
"Port 5000 is already in use"	Change PORT=5001 in .env and change the proxy in frontend/vite.config.js to http://localhost:5001.
Assistant gives only fixed-style answers	No API key is set, so the offline engine is used. Add GEMINI_API_KEY in .env and restart the backend.
Gemini key added but answers are still offline	The key is wrong or the free quota is used up. The dashboard chip shows the active AI mode.
PowerShell blocks npm scripts	Run once: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned, or use Command Prompt instead.

9. Viva questions and answers

Q. What is the MERN stack?

MongoDB (database), Express.js (backend framework), React (frontend library) and Node.js (JavaScript runtime for the server). All four use JavaScript.

Q. Why MongoDB instead of MySQL?

CRM data is flexible: leads have notes and AI results of different shapes. MongoDB stores these naturally as nested documents, and Mongoose still gives schemas and validation.

Q. How is login made secure?

Passwords are hashed with bcrypt before saving. On login the server creates a JWT token valid for 7 days. Every protected request sends the token, and middleware verifies it before running the route.

Q. How does the AI score a lead?

It combines stage, budget, source, deal value, job title, contact details, note sentiment and time since last contact into a 0-100 score, with reasons. With an API key, Gemini or OpenAI scores it instead.

Q. What if the internet or API key is not available?

The system automatically falls back to the offline rule-based engine, so every feature still works.

Q. How is the forecast calculated?

Each open deal value is multiplied by its AI score as a probability and summed. This is called a weighted pipeline forecast.

Q. What is sentiment analysis?

Finding the emotional tone of text. Here it tags each call note as positive, neutral or negative, and that affects the lead score.

Q. How does one user's data stay private?

Every lead, task and activity stores an owner id, and every database query filters by the logged-in user's id.

Q. What is a REST API?

A set of URLs where each HTTP method means an action: GET reads, POST creates, PUT/PATCH updates and DELETE removes data. React talks to the backend only through these.

Q. What is the role of React Router and Context?

React Router switches pages without reloading. AuthContext keeps the logged-in user available to all components.

Q. How does drag and drop work on the pipeline?

It uses the HTML5 drag-and-drop events. On drop, the app updates the screen immediately and calls PATCH /leads/:id/status. If the call fails, the change is undone.

Q. Future scope?

Sending emails directly with Nodemailer, WhatsApp follow-ups, CSV import, team roles, and training a machine learning model (e.g. logistic regression) on won/lost history to replace the rule-based scorer.